

Parallel Loop Subdivision on CUDA

Anoushka Naidu (anaidu), Tim Wang (twang3)

April 28, 2026

1 Links

1.1 Website

www.andrew.cmu.edu/user/anaidu/

1.2 Code

github.com/anoushkanaidu22/15418-loop-subdivision

2 Summary

We designed and implemented a fully parallel GPU-accelerated version of the Loop Subdivision algorithm using C++ and CUDA on an NVIDIA GeForce RTX 2080. Our goal was to overcome bottlenecks inherent to global mesh restructuring, specifically the exponential growth in complexity that comes with recursive surface changes. The geometry smoothing parts of the algorithm are inherently data parallel, but the topological restructuring, specifically the dynamic creation of vertices and the rebuilding of mesh connectivity, is challenging for parallel hardware because of race conditions and irregular memory access patterns. Instead of using pointer-based mesh structures normally used in sequential processing (for example halfedge data structure), our implementation uses a sort-scan strategy implemented with the NVIDIA Thrust library. We're essentially reframing the problem of topological connectivity as a series of global parallel sort and scan operations, and by doing this we can build up mesh adjacency lists and identify unique edges entirely on the GPU, which eliminates a major CPU bottleneck. We were able to greatly surpass our goal speedup of 10x with our implementation.

3 Background

Loop Subdivision is a standard algorithm in computer graphics for generating smooth surfaces from triangle meshes. It was created by Charles Loop in 1987 and is performed by moving original vertices to new positions, creating new vertices on edges, and updating connectivity. The limit surface of the algorithm is C^2 continuous (curvature continuous) everywhere except at extraordinary vertices (vertices with a valence other than 6), where it is C^1 continuous.

The algorithm happens in iterations, where each iteration quadruples the number of triangles in the mesh. This exponential growth (4^k faces after k iterations) means that a mesh of 10,000 faces creates a workload of millions of faces after just a few passes. A single iteration consists of two main components: geometry updates and topology updates.

The geometry updates involve calculating new positions for vertices based on local masks, which are weighted averages of a vertex's immediate neighbors. Every vertex v from the original mesh (referred to as an even vertex) is shifted to a new position v' . The new position is a weighted sum of the original position and its immediate 1-ring neighbors. Let k be the valence of vertex v (the number of neighbors it has). The new position is given by:

$$v' = (1 - k\beta)v + \beta \sum_{j=1}^k n_j$$

where n_j are the neighboring vertices. The weighting factor β is chosen to ensure smoothness continuity. According to Loop's original formulation (and standard PBR implementations), β is defined as:

$$\beta = \begin{cases} \frac{3}{16} & \text{if } k = 3 \\ \frac{1}{k} \left(\frac{5}{8} - \left(\frac{3}{8} + \frac{1}{4} \cos \frac{2\pi}{k} \right)^2 \right) & \text{if } k > 3 \end{cases}$$

This formula ensures that the limit surface behaves predictably regardless of how many edges connect to a vertex.

For every edge (u, v) shared by two triangles (u, v, a) and (u, v, b) , a new vertex e is created at the midpoint (referred to as an odd vertex). Its position is a weighted average of the edge endpoints (weighted $3/8$ each) and the two opposite vertices in the adjacent triangles (weighted $1/8$ each):

$$e = \frac{3}{8}(u + v) + \frac{1}{8}(a + b)$$

This mask ensures that the new vertex lies on the smooth limit surface defined by the surrounding patch.

We have to handle edges and vertices on the boundary of the mesh specially, where a boundary edge is defined as an edge that has only one adjacent face. To prevent the mesh from shrinking away from its borders, we use cubic B-spline rules for boundaries:

- **Odd Boundary Vertex:** Simply the midpoint of the edge:

$$e = \frac{1}{2}(u + v)$$

- **Even Boundary Vertex:** A weighted average of the vertex and its two boundary neighbors:

$$v' = \frac{3}{4}v + \frac{1}{8}(n_{\text{left}} + n_{\text{right}})$$

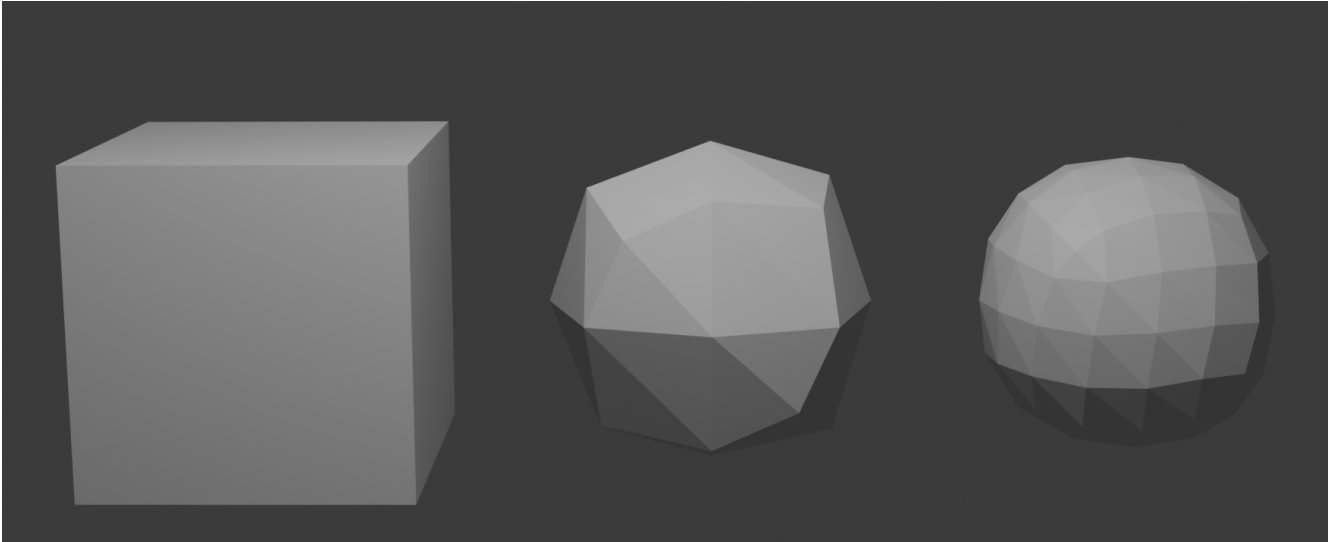


Figure 1: Two iterations of Loop subdivision on a cube.

We also have our topology updates. For every iteration, the algorithm splits every edge in the mesh to insert a new odd vertex. Then, every original triangle (u, v, w) is deleted and replaced by four new triangles: three corner triangles formed by an old vertex and two new edge vertices, and one center triangle connecting the three new edge vertices.

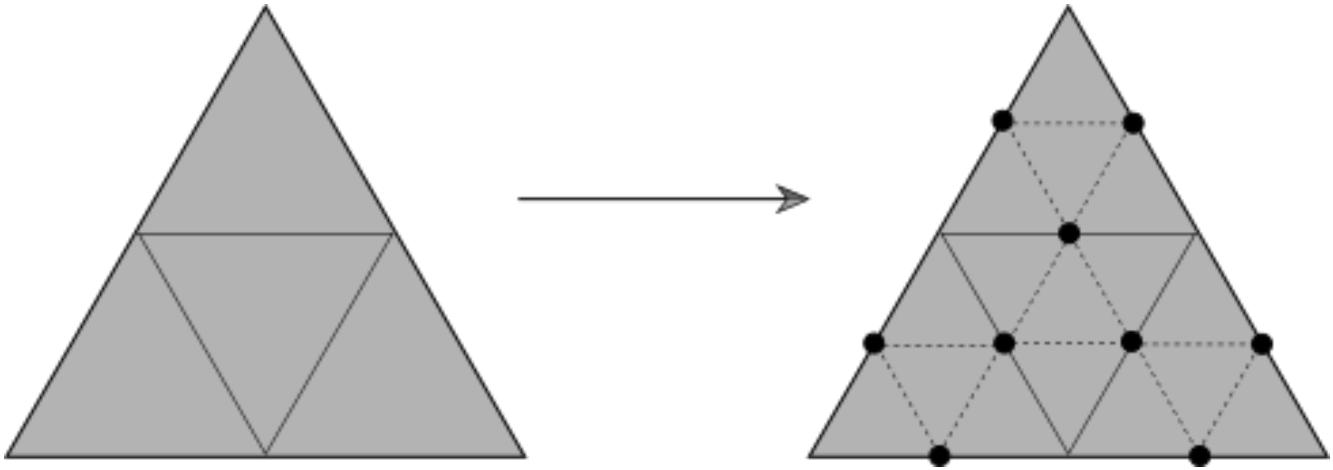


Figure 2: Topology Updates. https://www.pbr-book.org/3ed-2018/Shapes/Subdivision_Surfaces

Parallelizing this process is hard because mesh connectivity is irregular. Unlike a pixel grid where every element has exactly four neighbors, a mesh vertex can have any number of neighbors (valence). This irregularity makes it difficult to map mesh data to fixed-width SIMD registers of GPU threads efficiently without causing warp divergence or uncoalesced memory accesses.

Additionally, the algorithm is dynamic. We can't just update values in places, we have to allocate new buffers for vertices and faces that did not exist before, managing dependencies between the old topology and the new geometry.

Additionally, a point of variance for many 3D algorithms is the representation of the 3D object. Whether this be using implicit vs explicit geometric representations, we want to choose the representation that best fits the nature of the algorithm: specifically leveraging the local operations. We decided on an explicit representation due to its ability to easily query vertices and faces, as well as its flexibility in reasoning about complex meshes.

4 Approach

We implemented two different version of the Loop Subdivision algorithm. First, we implemented a reference sequential implementation on CPU to serve as a correctness baseline, and an optimized parallel implementation on the GPU using CUDA. We also implemented an OBJ parser and OBJ writer.

4.1 Sequential Implementation (CPU Baseline)

Our sequential implementation aims to provide a baseline for speedup against our parallel approach that will follow. Essentially what we are looking for is a purely sequential, bare bones implementation of the algorithm with minimal explicit optimization that would vastly change neither the asymptotic nor real-timed runtime.

The algorithm in our implementation proceeds in several phases. First, we perform topology analysis by constructing per-vertex neighbor lists and an edge-to-face map. These adjacency structures allow us to distinguish interior edges from boundary edges and to determine the number of neighbors, or valence, of each vertex. This information is required to apply the Loop subdivision rules correctly.

Next, we update the positions of the original vertices (even vertices). For each vertex, we examine its one-ring neighborhood and compute a weighted average of its neighbors. Interior vertices are moved toward a smooth blend of all their neighbors, while boundary vertices use a simpler rule that only involves their two boundary neighbors. This step smooths the mesh while still preserving the overall shape.

We then create new vertices (odd vertices) on each edge. For boundary edges, the new vertex is placed at the midpoint of the edge. For interior edges, the new point is computed using both the edge endpoints and the two vertices opposite that edge in the adjacent faces. This ensures that the surface remains smooth across face boundaries.

Finally, we rebuild the mesh connectivity. Each original triangle is replaced by four smaller triangles: three corner triangles, each formed from one original vertex and two adjacent edge vertices, and one central triangle formed from the three new edge vertices. After this reconstruction step, the mesh has a denser set of vertices and four times as many faces as before the iteration.

By repeating these phases for multiple iterations, the mesh becomes progressively smoother and converges toward a visually smooth surface, with high continuity in the interior and reasonable smoothness along boundaries.

4.2 Parallel CUDA Implementation



Figure 3: Smoothing as a result of 2 iterations of Loop subdivision.

Our parallel implementation targets the NVIDIA GPU architecture. We knew that just using our sequential C++ code would perform very poorly. GPU kernels can't efficiently allocate memory dynamically, making `std::vector::push_back` impossible, and pointer-chasing structures like linked lists cause severe memory divergence. So to achieve high performance, we redesigned the entire pipeline to work well on the GPU, relying on flat arrays and parallel sorting primitives.

4.2.1 Data Structures

To maximize memory bandwidth utilization, we replaced all array-of-structures layouts with structure-of-arrays layouts. On GPUs, memory coalescing is very important, so we want consecutive threads to access consecutive memory addresses. Therefore, instead of an array of `Face` objects, we use flat arrays of integers. For adjacency, we used the *Compressed Sparse Row* (CSR) format. This means we had two arrays: `adj_indices`, a huge integer array containing all neighbors for all vertices packed contiguously, and `adj_offsets`, an array where entry i points to the start of vertex i 's neighbors in the indices array. This allows any thread to find the neighbors of any vertex in $O(1)$ time with fully coalesced memory reads, completely eliminating the pointer chasing overhead of the CPU implementation.

4.2.2 Mapping to Hardware Architecture

To efficiently utilize the NVIDIA TRX 2080, we mapped mesh subdivision directly to the GPU's SIMT hardware hierarchy. We adopted a fine-grained decomposition strategy where specific mesh primitives were assigned to

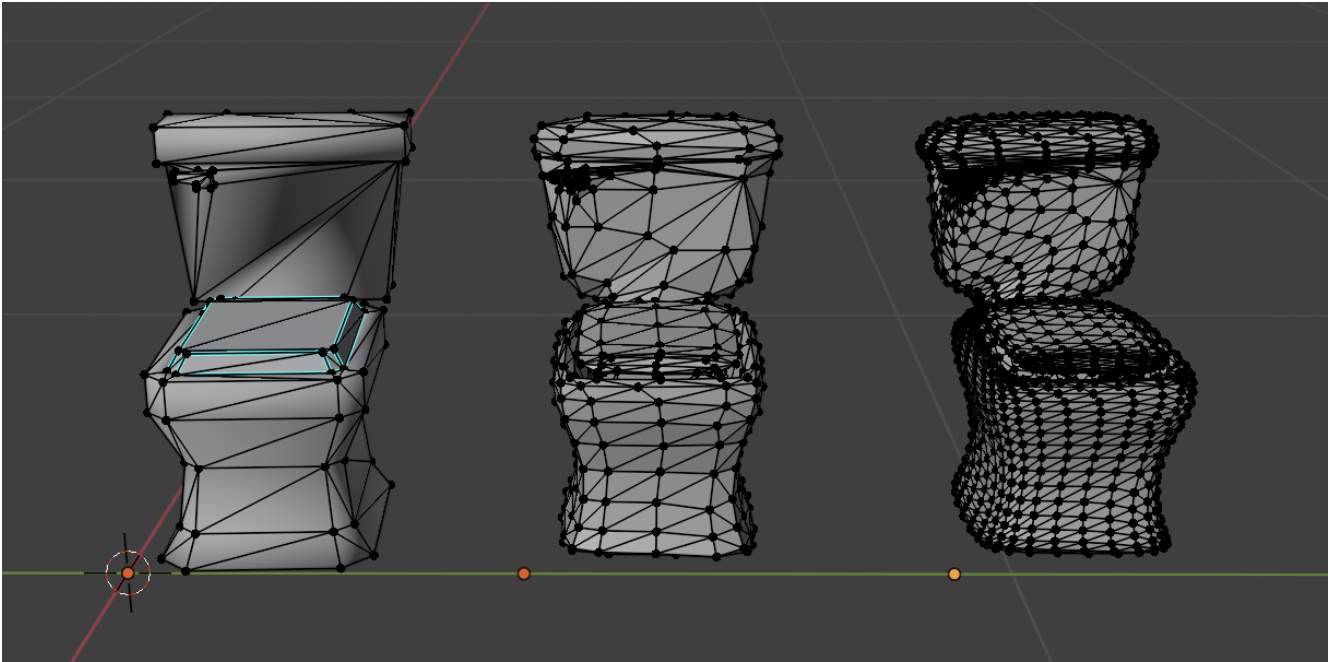


Figure 4: Visualization of the geometry changes over 2 iterations in Blender.

individual CUDA threads, allowing us to expose the maximum amount of parallelism to the hardware.

For the topology generation phase, we mapped one thread to one face. Each thread operates independently to read its face’s indices and emit three directed edge keys into global memory. This mapping aligns with the massive throughput capabilities of the GPU. As millions of faces are processed, the large number of threads allows the hardware warp scheduler to effectively hide the latency of global memory writes.

For the vertex smoothing phase, we transitioned the mapping so that one thread corresponds to one vertex. The thread ID is a direct index into our vertex buffers. This 1-to-1 mapping is really important for memory performance because it ensures that thread within a single warp (threads t to $t+31$) access logically contiguous memory addresses ($V[t]$ to $V[t+31]$). This pattern allows the GPU’s memory controller to coalesce these 32 requests into a minimal number of cache line transactions, maximizing bandwidth utilization.

For the edge splitting phase, we utilized the results of our topological sort to map one thread to one unique edge. This was really important because a naive per-face mapping would result in redundant calculations where shared edges are processed twice. By mapping threads to *unique* edges, we ensure computational efficiency. We configured our kernel launches using 1D grids with a block size of 256 threads. This block size was empirically selected to maximize occupancy on the RTX 2080’s SMs, ensuring that there are enough active warps to hide arithmetic latency.

Finally, we had to account for the hardware reality of warp execution. In CUDA architecture, thread within a warp execute in lock-step. Loop subdivision introduces inherent irregularity because vertices have varying valences. In our smoothing kernel this manifests as warp divergence since if one vertex in a warp has 6 neighbors and another as 12, the entire warp must wait for the longest loop to complete, with some threads idling. While this is a theoretical efficiency loss, we determined that the statistical convergence of mesh valences (averaging around 6) made this divergence acceptable compared the significant overhead of pre-sorting vertices by valence to enforce uniformity.

4.2.3 Sort-Scan

The biggest design challenge was constructing the topology. Specifically, constructing the adjacency list and unique edge list on the GPU without race conditions was difficult. A naive parallel approach would involve launching a kernel where each thread processes a face and adds neighbors to a list using `atomicAdd`, but this approach is not good. You would have to know the size of the neighbor lists beforehand to allocate memory, and for high-valence vertices, the serialization caused by atomic contention would destroy parallelism.

So instead of atomics, we implemented a sort-scan strategy by using the NVIDIA Thrust library. The inspiration for this was that many topological problems can be transformed into sorting problems. If we want to group all neighbors of vertex i together, we don't need to insert them into a list one by one, we can just generate a list of *all* edges in the mesh and sort them by their starting vertex.

To build the adjacency list in parallel, we launch an 'expansion' kernel where each thread processes one face and writes out six directed edges (two for each physical edge) into a large scratch buffer. We then use `thrust::sort` to sort this huge buffer on the source vertex ID. After sorting, all edges originating from Vertex 0 are contiguous in memory, following by edges from Vertex 1, and so on. We then use `thrust::lower_bound`, which is a parallel binary search, to find the indices where the source vertex changes. These indices become our `adj_offsets` array.

Finally, we have a simple copy kernel that populates the `adj_indices` array. This entire process build a perfect, duplicate free adjacency graph really efficiently, and it scales linearly with the number of edges, completely avoiding the need for locks or atomics.

We applied a similar strategy to identify unique edges for Phase 2. The challenging part here is that edge (u, v) and edge (v, u) refer to the same physical edge but appear as different data entries. To resolve this, we generate a key for every edge:

$$(\min(u, v), \max(u, v)).$$

We write these keys into a buffer and sort them. To implement this efficiently on the hardware, we packed these vertex pairs into CUDA `int2` vector types to ensure coalesced memory accesses. We then utilized `thrust::sort_by_key` with a custom comparator, which allows us to sort the edge keys while simultaneously reordering the associated face metadata needed for the reconstruction phase. After sorting, identical edges sit adjacent to each other in memory. We then run a custom `unique` kernel that scans this sorted array. When a thread identifies a key that is different from its predecessor, it marks a new unique edge and assigns it a global index.

This sort-scan pass also nicely lets us build the connectivity map. As we process the sorted edges, we can look at the metadata associated with each key to find the two faces sharing the edge and the opposite vertices required for the math mask. This allows us to build the Face-to-Edge mapping table in a single pass, enabling the final face reconstruction kernel to run without any searching.

4.2.4 Iterative Execution

A major potential bottleneck is always data movement. A 'hybrid' approach where the GPU performs the math but the GPU rebuilds the topology would require copying the entire mesh data back and forth for every iteration. Given that the mesh size quadruples each time, this transfer time would rapidly dwarf the computation time, capping speedup regardless of how fast the GPU is.

So to achieve our target speedup, we implemented a fully GPU-resident loop using a ping-pong buffering strategy. We allocate two distinct sets of mesh buffers on the GPU: `InputMesh` and `OutputMesh`. In iteration i , our kernels read strictly from `InputMesh` and write new geometry and topology to `OutputMesh`. At the end of the iteration, we do not download the data. Instead, we free the internal buffers of `InputMesh` and swap the pointers, so that `OutputMesh` becomes the input for iteration $i + 1$. This ensures that the mesh data remains in high-bandwidth VRAM for the entire subdivision process. The CPU's role is reduced to just orchestrating kernel launches, completely removing the PCIe bus and CPU processing capability from the critical path.

4.2.5 Robustness

Often times, mesh data is not ideal, and we wanted to be able to cleanly handle this. One significant issue we encountered was handling non-manifold geometry, specifically edges shared by three or more faces. In a standard Loop implementation, an edge is assumed to have exactly two adjacent faces (or one, if on a boundary). However, complex meshes like the toilet model in our dataset contained non-manifold edges. Our initial implementation naively checked only for a single "twin" face. If a third face shared the edge, its connectivity data remained uninitialized, leading to "exploding" spikes (as seen in Figure x) where vertices were rendered at garbage coordinates. To solve this, we modified our edge-identification kernel to implement a `while` loop that iterates through all sorted keys corresponding to a specific edge. This ensures that every single face attached to that edge receives the correct

unique edge index in the Face-to-Edge map. While the geometric smoothing rule still only uses two faces (clipping the others for the math calculation), this topological fix prevents the mesh from tearing or crashing, ensuring a stable visual output even for messy inputs.



Figure 5: Results when we did not handle non-manifold geometry.

We also had to address the issue of polygons with more than three sides. Loop Subdivision is mathematically defined strictly for triangular meshes; the rules for splitting faces and weighting vertices assume a valence-3 connectivity for faces. To handle general polygonal meshes (like quads), our system includes a mandatory pre-processing step in the OBJ parser. When loading a mesh, we detect any face with more than 3 vertices. We then automatically triangulate these faces on the fly before uploading the data to the GPU. This ensures that the CUDA kernels always receive a clean, triangulated input, simplifying the kernel logic and maximizing SIMD efficiency by avoiding branch divergence that would arise from handling variable face sizes inside the critical loop.

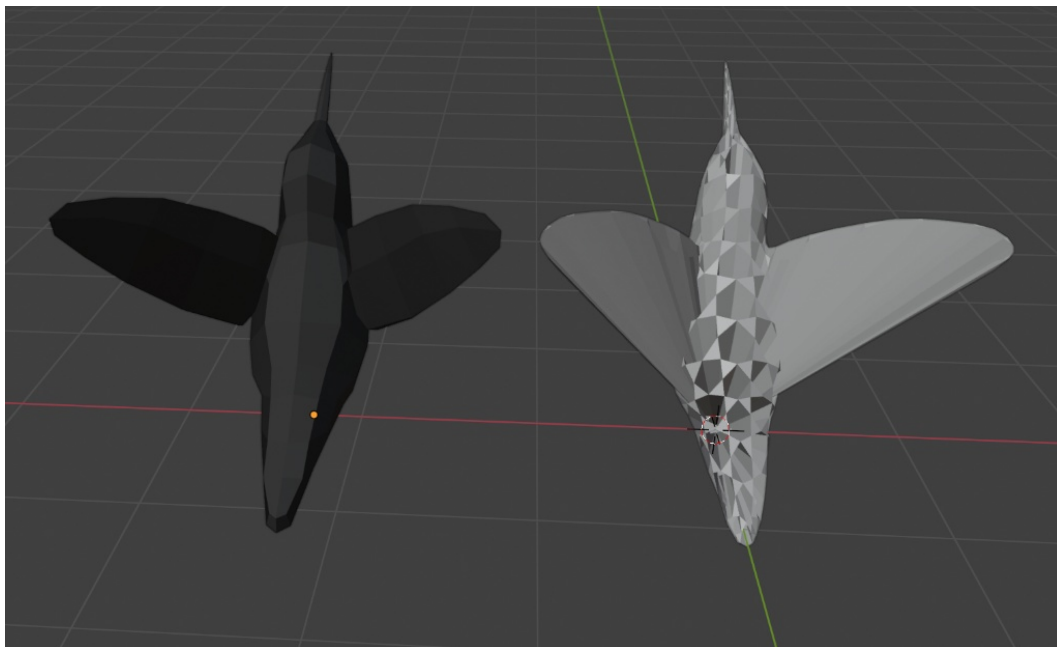


Figure 6: Results before handling meshes with faces with greater than 3 vertices. This mesh in particular was a quad mesh.

5 Results

We evaluated our implementation on an NVIDIA GeForce RTX 2080 GPU. We captured the time from the moment the mesh is loaded into memory to the completion of the final iteration. This includes all GPU memory allocations, the Thrust sort/scan topology building, and the execution of the math kernels. We excluded disk I/O time (loading and saving OBJ files) to isolate the performance of the algorithm itself.

5.1 Performance Comparison

We tested our implementation on a dataset of meshes with varying complexity to characterize scaling behavior. The results for a single iteration of subdivision are presented below.

Mesh	Faces	Edges	Verts	Seq. Time (s)	CUDA Time (s)	Speedup
Cube	8	18	12	0.000171	0.000563	0.30x
Toilet	228	342	118	0.000451	0.000484	0.93x
GTR Car	14,762	21,330	6,852	0.021652	0.000908	23.85x
Godzilla	25,946	38,919	14,081	0.039032	0.001129	34.57x
Lion	47,214	141,612	141,642	0.135979	0.002340	58.11x
Rat	57,216	171,648	171,648	0.163295	0.005670	28.80x

Table 1: Comparison of Sequential vs. CUDA runtime for 1 iteration of Loop Subdivision.



Figure 7: Meshes we tested the performance of our implementation on in order of increasing number of vertices. Some meshes were scaled to fit in one picture.

5.2 Analysis

The performance data reveals that for very small meshes like the Cube and Toilet, the CUDA implementation is actually slower than the CPU, achieving only 0.30x and 0.93x speedup. This is likely because the fixed overhead of CUDA API calls combined with the setup time for `thrust::sort` (which allocates temporary buffers and invokes kernels) is roughly 0.5 milliseconds. When the total computation takes less than 0.2 milliseconds on the CPU, the GPU simply cannot compete. This confirms that our approach is optimized for throughput, not latency; the GPU needs a sufficient workload to amortize its startup costs. The overhead of our CUDA implementation means that for small meshes, the sequential implementation is superior.

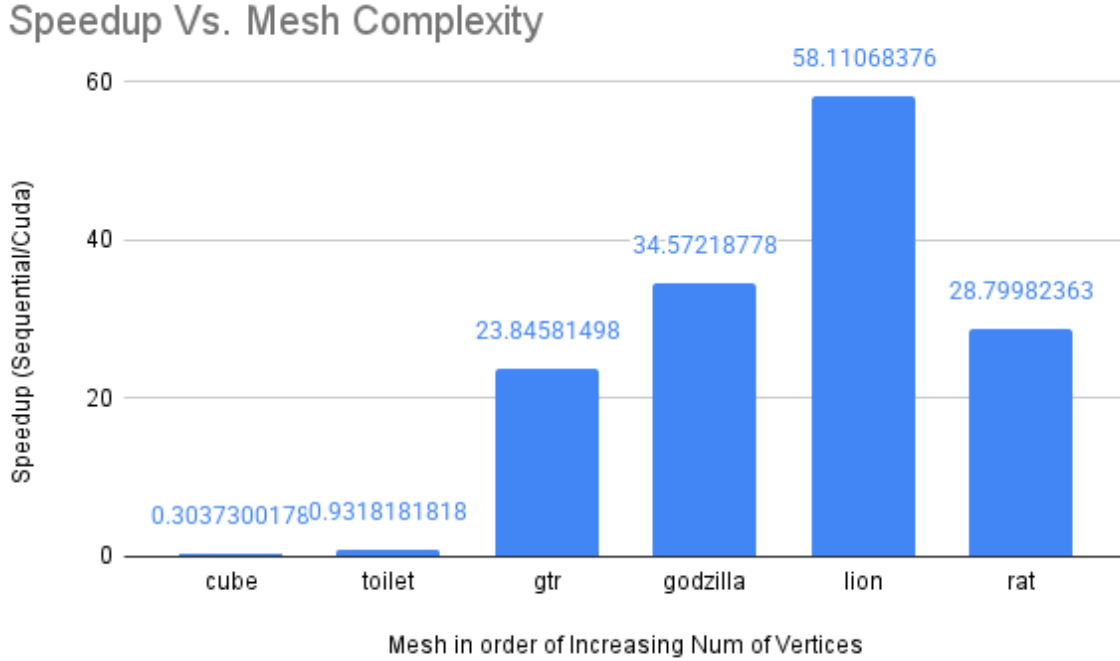


Figure 8: Speedup achieved relative to mesh complexity.

As the problem size grows, the speedup increases dramatically. For medium-sized meshes like the GTR Car and Godzilla, we see a jump to 23x and 34x speedups. In this size range, the number of threads is sufficient to hide some memory latency, and the GPU’s massive memory bandwidth begins to do much better than the CPU’s cache-dependent performance. The sorting operations, which are $O(N \log N)$, are executed by thousands of cores in parallel, making them significantly faster than the CPU’s sequential map building.

The Lion mesh hit the sweet spot so to speak for our implementation, achieving a massive 58x speedup. At 47,000 faces, we have enough work to fully saturate the GPU’s streaming multiprocessors (SMs). The overhead of sorting becomes negligible compared to the brute-force speedup of processing 47,000 geometric items in parallel versus sequentially.

However, we observed an interesting anomaly with the Rat mesh. Despite being larger than the Lion mesh, the speedup drops significantly from 58x to 28x. We hypothesize two potential causes for this regression. First, it could be because of topology irregularity and divergence. If the Rat mesh has high-valence vertices (vertices connected to 10+ neighbors) concentrated in certain areas, the adjacency processing kernels will suffer from warp divergence. In a GPU warp of 32 threads, if one thread has to loop 15 times (valence 15) while others loop 6 times (valence 6), the entire warp waits for the slowest thread, reducing SIMD efficiency. Second, the memory access patterns could be problematic. The layout of vertices in the Rat OBJ file might be less spatially coherent than the Lion. Since our adjacency list relies on indirect memory access (`vertices[adj_indices[i]]`), poor locality leads to uncoalesced global memory reads. If the neighbors of a vertex are scattered randomly across the vertex buffer rather than being clustered, the GPU cache hit rate drops, and the algorithm becomes memory-bound.

5.3 Component Breakdown Analysis

To better understand where our GPU implementation does well and where it is the slowest, we analyzed the breakdown of execution time between the “Topology” phase (building adjacency and edges via Sort-Scan) and the “Geometry” phase (math kernels). We were able to isolate these components by looking at initialization time (topology) and total time (so geometry time it total - initialization).

Component Analysis

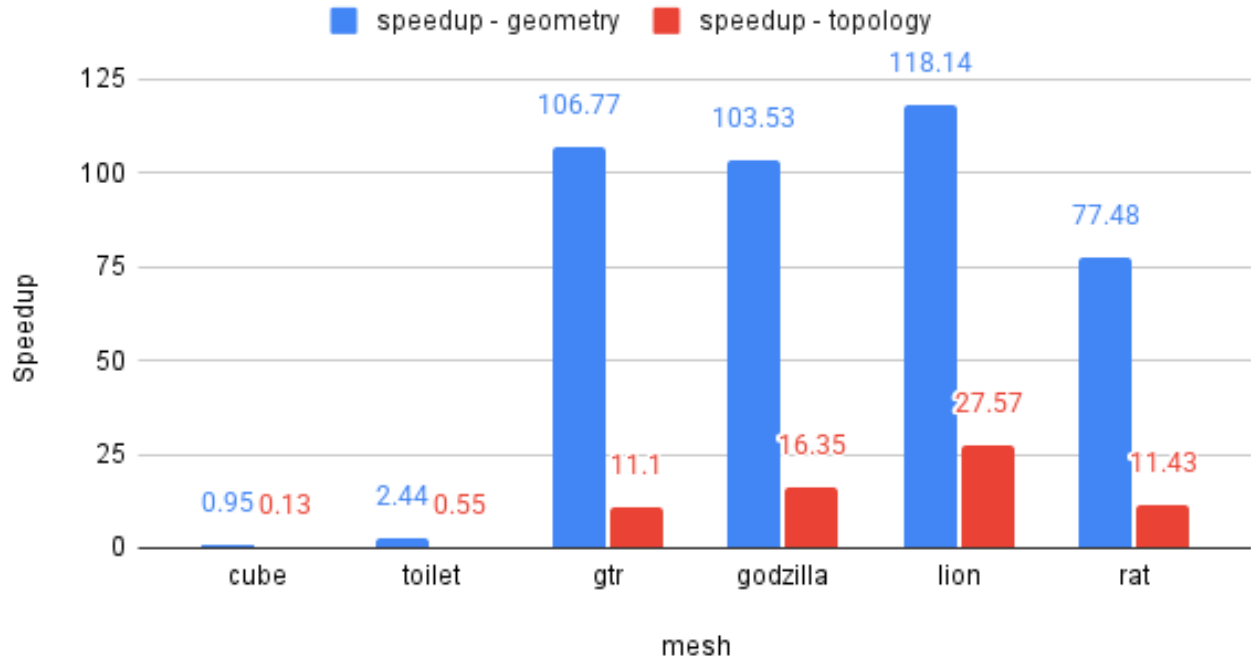


Figure 9: Component analysis of speedup for the geometry and topology updates across different meshes.

For the Lion mesh, the GPU spent 1.55ms on Topology and only 0.79ms on Geometry. Comparing this to the CPU baseline reveals a big disparity. The CPU spent approximately 42.7ms on Topology and 93.3ms on Geometry. This means our GPU implementation achieved a 27.5x speedup on Topology, but a much larger 118x speedup on Geometry. This confirms that the math portion of Loop Subdivision is effectively extremely parallel and benefits massively from the raw FLOPS of the GPU. The topology phase, while significantly accelerated by our Sort-Scan strategy, remains the bottleneck due to the memory-bandwidth-intensive nature of global sorting. The topology phase involves moving data across the entire GPU to group edges, whereas the geometry phase is largely local, reading neighbors that are often cached.

Comparing the Rat mesh to the Lion mesh through this lens offers a clear explanation for the rat anomaly we observed. For the Rat, the geometry speedup dropped to 77x and the topology speedup dropped to 11x. While the geometry speedup decline likely reflects poorer memory locality for vertex reads (uncoalesced access), the collapse in topology speedup (from 27x to 11x) is the main problem. This drop indicates that the Sort-Scan pipeline is very sensitive to the specific graph structure of the input. The Rat mesh likely contains edge cases like an uneven distribution of edge keys or high-valence nodes that trigger worst-case behaviors in the `thrust::sort` or `thrust::scan` primitives, greatly increasing the time required to build the connectivity graph.

6 Failed Approaches & Fixes

We didn't initially start with sort-scan solution. We first tried several other strategies that failed/weren't as good until we landed on it.

Our initial design was a hybrid CPU-GPU approach. We thought that since topology building is hard on the GPU, we should just keep it on the CPU. We implemented a pipeline where the CPU would build the adjacency maps, upload them to the GPU, run the math kernels, and then download the results back to the CPU to rebuild connectivity for the next iteration. While this did successfully produce correct meshes, the performance was really bad, sometimes 2x slower than CPU baseline even for large meshes. We found that most of the time was spent in

`cudaMemcpy` and CPU execution. This failure taught us that Loop Subdivision is bandwidth-bound, and moving the exponentially growing mesh data across the PCIe bus completely negates any advantage gained from parallelizing the math. This forced us to commit to a fully GPU resident pipeline.

We also attempted a naive atomics approach to build the topology on the GPU. We tried to launch a kernel where each thread processed a face and used `atomicAdd` to increment neighbor counts and write indices into a global buffer, but this approach failed for both correctness and performance reasons. Correctness was an issue because we couldn't easily predict the memory size requirements for the neighbor lists without a separate pre-pass, leading to buffer overflows. Performance was poor because `atomicAdd` serializes execution when many threads try to write to the same location. For vertices with high valence, this contention created a bottleneck that stalled thousands of threads. This failure led us to the Thrust sort-scan approach, which is deterministic, lock-free, and scales predictably.

7 Limitations & Future Work

Despite the strong performance, our implementation has limitations that would need to be addressed in next steps. The most significant limitation is memory usage. Loop subdivision quadruples the face count with every iteration. Since we pre-allocate destination buffers on the GPU to avoid dynamic allocation, we hit VRAM limits much faster than system RAM limits on a CPU. A mesh with 1 million faces cannot undergo 3 iterations on an 8GB card without crashing. A future improvement would be to implement streaming subdivision or use CUDA Unified Memory to oversubscribe VRAM, allowing the driver to page data to system RAM automatically, although with a performance penalty.

Another limitation is the handling of boundary conditions. Our implementation detects boundaries by checking if an edge has only one adjacent face. However, Loop Subdivision has specific rules for crease edges and corner vertices to maintain sharp features. Our current kernel uses a simplified boundary rule (simple midpoint averaging). While this prevents the mesh from shrinking, it does not support tagged sharp features, meaning a sharp cube eventually becomes a smooth blob rather than a cube with rounded edges. Implementing full crease support would require an additional "Tags" buffer to store per-edge metadata and more complex conditional logic in the math kernels.

Finally, we observed that warp divergence remains a concern for meshes with highly irregular valences. In the kernels that iterate over neighbors (specifically `reposition_even_kernel`), we assume a somewhat uniform valence distribution. If the mesh contains super-nodes (vertices with 20+ neighbors) alongside regular vertices (valence 6), we incur significant thread divergence. A potential optimization would be to sort vertices by valence before processing, ensuring that warps process vertices of similar complexity, thus keeping the instruction stream uniform and maximizing SIMD utilization.

8 Conclusion

We successfully implemented an efficient parallel Loop Subdivision system on the GPU. By using parallel primitives like Sort and Scan instead of sequential data structures, we achieved a high of **58x speedup** over the CPU baseline.

9 Work Distribution

- Both: Project proposal, final project writeup, approach and design of solution
- Anoushka (70%): midpoint post, sequential implementation, CUDA implementation, website
- Tim (30%): project idea, testing and benchmarking, timing code, procured meshes for testing

10 References

Below is credit for the meshes we used for testing:

- `cube.obj`: Blender library
- `toilet.obj`: Poly Pizza. n.d. Toilet Square. Accessed December 2025.
<https://poly.pizza/m/RTlsBgoX44>
- `GTR.obj`: Poly Pizza. n.d. Nissan GTR. Accessed December 2025.
<https://poly.pizza/m/a.HKctYAv2W>
- `mechagodzilla.obj`: Poly Pizza. n.d. Nissan GTR. Accessed December 2025.
<https://poly.pizza/m/6ljjFt431xK>
- `lion_head.obj`: Poly Haven. n.d. Lion Head. Poly Haven. Accessed December 2025.
https://polyhaven.com/a/lion_head
- `rat.obj`: Poly Haven. n.d. Lion Head. Poly Haven. Accessed December 2025.
https://polyhaven.com/a/street_rat

Figure 2 is from https://www.pbr-book.org/3ed-2018/Shapes/Subdivision_Surfaces
All other images were created by us.